SEMINAR REPORT
ON

Loan Default Risk Prediction using Random Forest

BY

Name : Prasad Bhalerao **RBT NO: RBT24CB026**

Third Year B. TECH
ACADEMIC YEAR: 2026-27

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING TATHAWADE,
PUNE - 411033

CERTIFICATE

This is to certify that the seminar report entitled **"Loan Default Risk Prediction using Random Forest"** submitted by Prasad Bhalerao (PRN No. RBT24CB026) is a record of bonafide work carried out under my supervision, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Business Systems of Savitribai Phule Pune University, during the academic year 2026-27.

Date:

Prof. Neeraj Sathwale	**Dr. K. P. Moholkar**	**Dr. A. M. Badadhe**
Seminar Guide	H. O. D. CSBS	Director, R.S.C.O.E. Pune

Acknowledgement

I express my sincere gratitude to my seminar guide, Prof. Neeraj Sathwale, Department of Computer Science and Business Systems, for the guidance, encouragement and constructive feedback given throughout this work.

I thank Dr. K. P. Moholkar, Head of the Department, and Dr. A. M. Badadhe, Director, for the facilities and academic support that made this seminar possible.

I am also grateful to the faculty and staff of the department, and to my family and friends, for their help and motivation.

Prasad Bhalerao
RBT24CB026
Department of Computer Science and Business Systems
JSPM's Rajarshi Shahu College of Engineering, Pune

Abstract

This seminar studies the Random Forest algorithm applied to credit risk, and reports a complete working implementation. The dataset is the public Kaggle “Give Me Some Credit” collection of 150,000 labelled borrower records with 10 numeric attributes. The target, SeriousDlqin2yrs, marks borrowers 90 or more days past due within two years — only 6.68% of records.

That imbalance makes accuracy unusable as a headline metric: always predicting the majority class scores 93.3% while catching nobody. The work therefore selects on the area under the precision-recall curve, treats undocumented placeholder codes explicitly, and chooses the decision threshold on validation data rather than assuming 0.50.

On 30,000 held-out records scored exactly once, the model reached ROC-AUC 0.8654 and PR-AUC 0.4014 against a 0.0668 no-skill baseline. The pipeline was deployed as a REST service with a web interface. The report also documents the limitations that make the system unfit for real credit decisions.

Keywords: Random Forest, Credit Risk Prediction, Class Imbalance, Precision-Recall Curve, Decision Threshold, Supervised Classification

INDEX

Sr. No.	Contents	Page No.
1	Acknowledgement	iii
2	Abstract	iv
3	List of Figures and Tables	v
4	Chapter 1: Introduction	1
5	Chapter 2: Literature Review	2
6	Chapter 3: Technology / Concept Overview	4
7	Chapter 4: Applications and Results	8
8	Chapter 5: Advantages and Limitations	13
9	Chapter 6: Future Scope	15
10	Chapter 7: Conclusion	16
11	References	17

LIST OF FIGURES AND TABLES

Figure	Title	Page
Fig. 3.1	System architecture of the implemented application	4
Fig. 3.2	How a Random Forest produces a prediction	5
Fig. 4.1	Dashboard of the deployed application	9
Fig. 4.2	Model information page, read live from saved metadata	10
Fig. 4.3	Predicted score distribution by actual outcome	11

Table	Title	Page
Table 2.1	Summary of literature review	2
Table 3.1	Key concepts and definitions	4
Table 3.2	Input features	6
Table 3.3	Data quality issues and treatment	6
Table 3.4	Validation comparison of configurations	6
Table 4.1	Held-out test metrics	11
Table 4.2	Per-class report and confusion matrix	11
Table 4.3	Effect of the decision threshold	12
Table 5.1	Advantages and limitations	13
Table 7.1	Technology stack	16

CHAPTER 1: INTRODUCTION

1.1 Overview

Credit risk prediction estimates how likely a borrower is to miss repayment. Traditionally an underwriter applied policy rules by hand. Machine learning instead fits a model to past borrowers whose outcomes are known, and learns the link between attributes and outcome.

This seminar studies that application through the Random Forest algorithm. Each record describes one borrower by 10 numeric attributes; the label states whether they were 90+ days past due within two years. It is supervised binary classification — the output is a category, not a quantity, and the groups come from labels rather than being discovered.

1.2 Need and Importance

Lenders extend credit under uncertainty. A default is a direct loss; a sound applicant wrongly refused is lost revenue. Better ranking of applicants by risk therefore has direct economic value, and manual assessment does not scale.

The problem is also statistically interesting. Only 6.68% of borrowers here are positive, and that imbalance breaks the most familiar metric: a model that always answers “no delinquency” scores 93.3% accuracy while catching nobody. Which metrics replace accuracy, and why, is a central theme of this report.

The same reasoning transfers to fraud detection, medical diagnosis and equipment failure — any rare-event prediction problem.

1.3 Objectives

- To understand the Random Forest algorithm: bagging, random feature selection, and how an ensemble yields a probability.
- To study the architecture of a complete prediction system, from raw data to a deployed service.
- To examine correct evaluation under class imbalance, and show why accuracy is inadequate.
- To implement the method on a real dataset and report measured results.
- To identify the advantages, limitations and ethical challenges of such systems.

CHAPTER 2: LITERATURE REVIEW

Breiman [1] introduced Random Forests. Combining many decorrelated trees reduces variance without raising bias, and generalisation error converges as trees are added. Two sources of randomness are responsible: bootstrap sampling of rows, and restricting each split to a random feature subset.

Saito and Rehmsmeier [4] were the most influential source here. ROC plots mislead under imbalance, because the false-positive rate is measured against a large negative population and barely moves. Precision-recall plots do not, since precision counts only what was flagged. This is why the model below is selected on PR-AUC.

Pedregosa et al. [2] document scikit-learn. Its RandomForestClassifier averages per-tree leaf class proportions — not the hard majority vote introductory treatments describe. That distinction is kept throughout.

Chawla et al. [5] survey imbalance remedies: resampling and class weights. Framing the problem as choosing an operating point on a trade-off curve, rather than fixing a broken model, informed the decision here to move the threshold instead of reweighting.

Table 2.1: Summary of literature review

Sr.	Author(s), Year	Focus	Key Findings
1	Breiman, 2001	Random Forest theory	Averaging decorrelated trees cuts variance; error converges as trees are added.
2	Pedregosa et al., 2011	scikit-learn	Reference implementation. Probability = mean of per-tree leaf proportions, not a vote.
3	Kaggle, 2011	Give Me Some Credit data	150,000 labelled records with a documented data dictionary.
4	Saito & Rehmsmeier, 2015	Imbalanced evaluation	ROC is over-optimistic under imbalance; PR is more informative.
5	Chawla et al., 2002	Resampling, cost-sensitive learning	Imbalance is a choice of operating point; reweighting shifts it rather than improving ranking.

Research gap.

The literature agrees that accuracy and ROC-AUC flatter a classifier when positives are rare, yet both are still widely reported. The standard remedies are often applied

without checking whether they improve ranking or merely relocate the operating point.

This work measures that. Class weighting is found to move the model along the precision-recall curve without lifting it, so explicit threshold selection is used instead. Separately, undocumented placeholder codes that much published work on this data imputes away are shown to carry real signal.

CHAPTER 3: TECHNOLOGY / CONCEPT OVERVIEW

3.1 Basic Concepts

Table 3.1: Key concepts and definitions

Term	Definition
Decision tree	Repeatedly splits data on one attribute. Terminal nodes (leaves) hold the training samples that reached them.
Gini impurity	$G = 1 - \Sigma\, pk^2$, where pk is the proportion of class k in a node. Zero when a node holds one class. Splits maximise the weighted reduction in G.
Bagging	Each tree trains on a sample drawn with replacement, so every tree sees different data. Averaging reduces variance.
Random Forest	An ensemble of 300 such trees, each split restricted to a random feature subset.
Target	SeriousDlqin2yrs — 1 if 90+ days past due within two years, else 0.
Precision	Of those flagged, the share truly delinquent. TP / (TP + FP).
Recall	Of the truly delinquent, the share found. TP / (TP + FN).
F1-score	Harmonic mean 2PR / (P + R); dominated by the weaker of the two.
ROC-AUC	Probability a random positive outranks a random negative. 0.5 is chance.
PR-AUC	Area under the precision-recall curve. No-skill baseline = the positive rate, here 0.0668.
Threshold	The probability above which a prediction is reported positive. Chosen after training.

3.2 Architecture

Training, evaluation and figure generation run offline and produce one saved artifact. The web service loads it at startup and scores requests; it never trains.

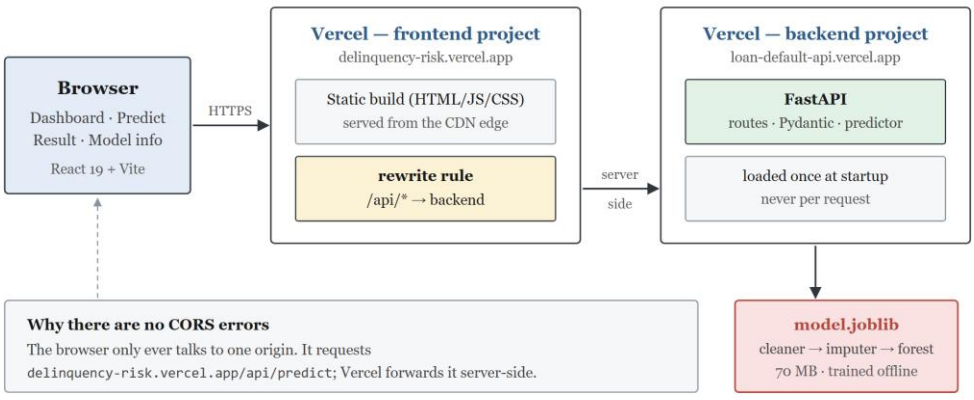


Figure 3.1: System architecture of the implemented application

The browser talks to one origin only — requests to /api are forwarded server-side, so no cross-origin request is ever made.

Training pipeline.

Six stages: load and validate against a schema; mark implausible values missing; split 60/20/20 stratified; compare five candidates on validation; select the threshold; score once on the test split.

Every fitted quantity — imputation medians, model parameters, the threshold — comes from train and validation only. The test split is held in reserve until the final stage. Stratifying matters at a 6.68% base rate: an unstratified split could leave the three parts with different class ratios, making their metrics incomparable.

3.3 How a Prediction Is Made

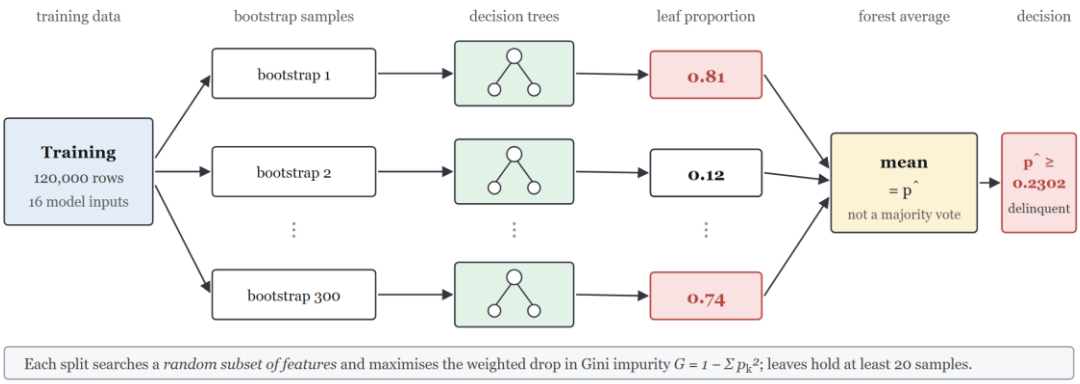


Figure 3.2: How a Random Forest produces a prediction

Each of the 300 trees routes the record to a leaf and reports that leaf's positive proportion. The forest averages those 300 values — scikit-learn averages per-tree probabilities rather than taking a majority vote. The mean is compared with the threshold of 0.2302 to give a class.

3.4 Features and Data Quality

Ten numeric attributes per borrower. No categorical encoding is needed, and because trees split on rank order, no scaling is either.

Table 3.2: Input features

Feature	Description
RevolvingUtilizationOfUnsecuredLines	Balances ÷ total credit limits
age	Borrower age in years
NumberOfTime30-59DaysPastDue	30–59 day late episodes, last 2 years
DebtRatio	Debt, alimony and living costs ÷ gross monthly income
MonthlyIncome	Gross monthly income (19.82% missing)
NumberOfOpenCreditLinesAndLoans	Open loans and credit lines
NumberOfTimes90DaysLate	90+ day late episodes
NumberRealEstateLoansOrLines	Mortgages and home equity lines
NumberOfTime60-89DaysPastDue	60–89 day late episodes, last 2 years
NumberOfDependents	Dependents excluding the borrower (2.62% missing)

Table 3.3: Data quality issues and treatment

Issue	Extent	Treatment
MonthlyIncome missing	29,731 (19.82%)	Median imputation + missing indicator
NumberOfDependents missing	3,924 (2.62%)	Median imputation + missing indicator
Counters coded 96 or 98	269 rows	Placeholders. Imputed with an indicator — these rows are 54.65% positive against a 6.68% base rate
Age recorded as 0	1 row	Impossible; treated as missing
Duplicate predictor rows	646 rows	Retained — plausibly distinct borrowers
Extreme ratios	up to 329,664	Retained — trees split on rank order

Preprocessing and the classifier are one scikit-learn Pipeline, and that composite object is what is saved. Serving code loads and calls it, so train-serve skew is structurally impossible. The classifier receives 16 columns built from the 10 features, because of the appended indicators.

3.5 Model Selection

Five configurations were fitted on train and compared on validation, ranked by PR-AUC.

Table 3.4: Validation comparison of configurations

Configuration	ROC-AUC	PR-AUC	Precision	Recall
baseline	0.8450	0.3481	0.5334	0.1870
regularised	0.8615	0.3820	0.5988	0.1556
balanced	0.8602	0.3776	0.2399	0.7197
balanced subsample	0.8602	0.3780	0.2805	0.6504
regularised with indicators	0.8630	0.3863	0.6290	0.1581

Raising the minimum leaf size to 20 lifted PR-AUC from 0.3481 to 0.3820 — the unrestricted forest was overfitting.

Class weighting raised recall from 0.1556 to 0.7197 but left PR-AUC flat at 0.3776. It moved the model along the curve without lifting it, so it was not used — a threshold does the same job visibly and without refitting.

Adding missing-value indicators won, at 0.3863. The threshold was then set on validation by maximising F1, giving 0.2302 rather than 0.50. The model was refitted on train + validation and scored once on test.

CHAPTER 4: APPLICATIONS AND RESULTS

4.1 Applications and Use Cases

Application scoring. A score at origination, combined with policy rules and affordability checks. Every applicant is assessed on the same basis in milliseconds. The system below demonstrates this directly.

Behavioural scoring. Rescoring an existing portfolio as payment behaviour accumulates, to catch accounts drifting toward delinquency early enough to intervene.

Collections prioritisation. Ranking delinquent accounts so finite recovery effort goes where it matters. Needs only the ranking, not a calibrated probability.

The technique generalises to any rare-event problem on tabular data: fraud detection, claim triage, churn, predictive maintenance, clinical risk scoring.

The benefits are consistency, scale and inspectability — identical criteria for every applicant, at volumes no manual process could reach, with the attributes the model relies on open to scrutiny.

4.2 Deployed Application

The model is deployed at https://delinquency-risk.vercel.app, with API documentation at https://loan-default-api.vercel.app/docs. Four pages: Dashboard, Predict, Result and Model info. All displayed values are fetched from the service at run time.

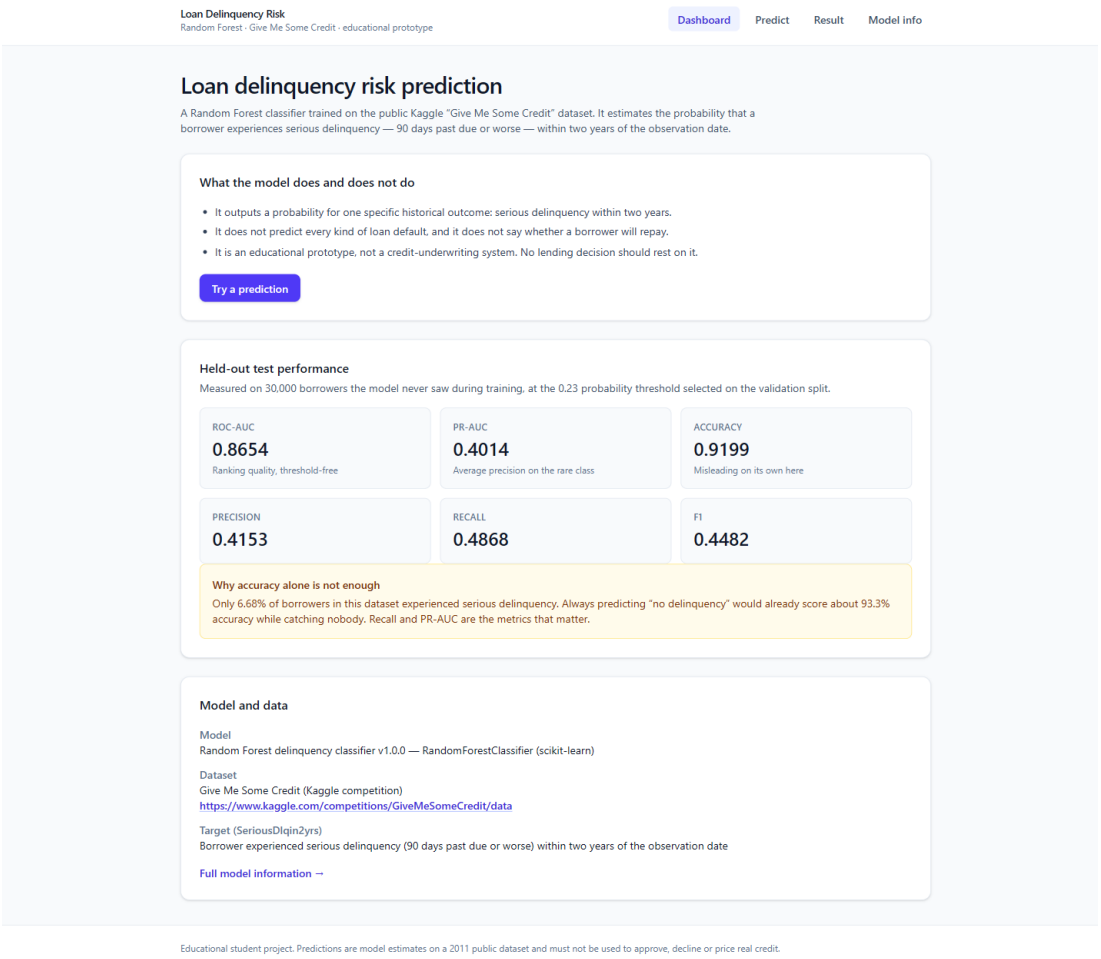


Figure 4.1: Dashboard of the deployed application

The dashboard states what the model does and does not do, and shows the majority-class baseline beside the accuracy so the latter cannot be mistaken for a strong result.

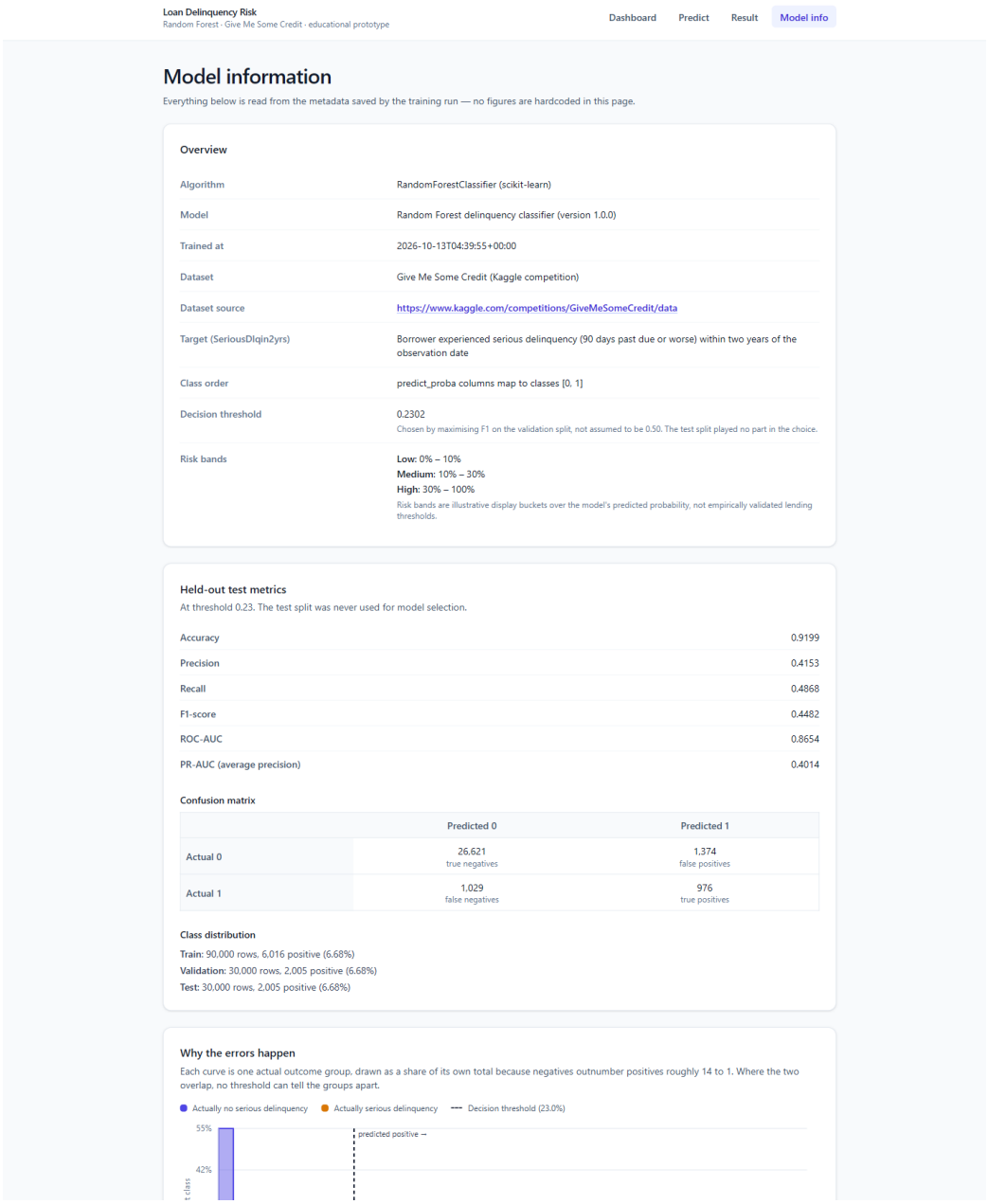


Figure 4.2: Model information page, read live from the saved metadata

The form exposes exactly the ten trained attributes with readable labels; the result page separates the probability, the thresholded class and an illustrative risk band.

4.3 Results

Measured on 30,000 held-out borrowers, scored once, at threshold 0.2302.

Table 4.1: Held-out test metrics

Metric	Value	Read it as
ROC-AUC	0.8654	Ranks a delinquent above a sound borrower 86.5% of the time
PR-AUC	0.4014	~6× the no-skill baseline of 0.0668
Recall	0.4868	Caught 976 of 2,005 real delinquencies
Precision	0.4153	42% of those flagged truly were
F1-score	0.4482	Positive class
Accuracy	0.9199	Below the 93.3% of always answering “no”

Table 4.2: Per-class report and confusion matrix

Class	Precision	Recall	F1	Support
0 — No delinquency	0.9628	0.9509	0.9568	27,995
1 — Delinquency	0.4153	0.4868	0.4482	2,005
Macro average	**0.6891**	**0.7189**	**0.7025**	**30,000**
Weighted average	0.9262	0.9199	0.9228	30,000
Confusion matrix	TN 26,621	FP 1,374	FN 1,029	TP 976

This is the imbalance problem in one table. The model scores 0.9568 F1 on the majority class and 0.4482 on the minority. The weighted average of 0.9228 looks impressive but is dominated by 27,995 negatives and says almost nothing about the class the system exists to find. Quote the macro average (0.7025) or the positive-class F1 — never the weighted one.

Why the errors occur.

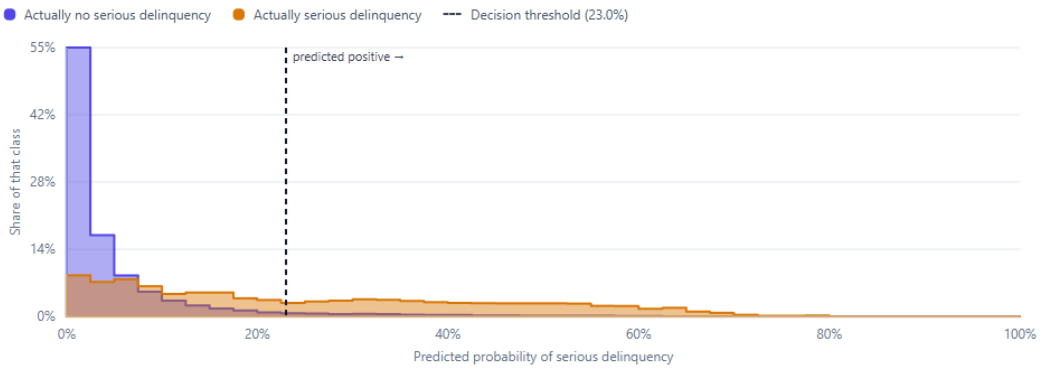


Figure 4.3: Predicted score distribution by actual outcome

Most borrowers who stayed current score near zero. Those who became delinquent are spread across the whole range rather than clustered high, so many fall inside the same region as healthy borrowers. That overlap — not a defect in the implementation — is what limits recall.

Table 4.3: Effect of the decision threshold

Threshold	Accuracy	Precision	Recall	F1	Flagged
0.1000	0.8469	0.2610	0.7047	0.3810	5,413
0.2000	0.9131	0.3884	0.5222	0.4454	2,696
0.2302	0.9199	0.4153	0.4868	0.4482	2,350
0.3000	0.9292	0.4650	0.3970	0.4283	1,712
0.5000	0.9374	0.6267	0.1566	0.2506	501

The same model behaves very differently depending on the cut-off. At 0.10 recall exceeds 0.70 but precision falls to 0.26 — three in four flags are false alarms. At 0.50 precision passes 0.62 but only one delinquency in six is found. Accuracy rises monotonically simply because the model answers “no” more often, which is a further reason not to trust it here.

Stability.

Five-fold cross-validation on train + validation, test excluded throughout, gives ROC-AUC 0.8632 ± 0.0053 and PR-AUC 0.3978 ± 0.0148. The test values sit within one standard deviation of both, so the result is not a lucky split.

ROC-AUC falls from 0.9329 in-sample to 0.8654 held-out, a gap of 0.0675. Some overfitting remains despite regularisation; it is reported rather than concealed.

CHAPTER 5: ADVANTAGES AND LIMITATIONS

Table 5.1: Advantages and limitations

Advantages	Limitations
Captures non-linear interactions without feature engineering	Data describes 2011 US consumers; current performance unknown
Robust to extreme values; no scaling or encoding needed	No date column, so out-of-time validation is impossible
Outputs probabilities, not just labels	Probabilities are uncalibrated; calibration was not evaluated
Threshold adjustable after training, without refitting	Threshold optimises F1, weighting both error types equally
Reports feature importances, keeping the model inspectable	Importance describes splits, not causation
Trains in ~1 minute; reproducible with a fixed seed	No protected attributes, so fairness cannot be tested

5.1 Key Limitations

Data, not algorithm. The records are US consumers around 2011 with no date column, so no out-of-time validation is possible and the sampling frame is undocumented.

Uncalibrated. A predicted 0.30 is a ranking score, not a claim that 30 in 100 such borrowers default.

Fairness untested. No race, sex or geography in the data. That is not protection — age is used, and income or utilisation can proxy for protected traits. Absence of measured bias is absence of measurement.

Threshold economics. Maximising F1 weights a missed default and a false alarm equally. Real lending does not; choosing properly needs loss-given-default and margin figures the dataset lacks.

5.2 Challenges Encountered

Undocumented codes. The codes 96 and 98 appear in 269 records with no explanation. Those rows are 54.65% positive against a 6.68% base rate, so imputing them away would discard signal while asserting clean payment histories. Imputing with a retained indicator lets the model learn the value was unavailable, without guessing what the codes mean.

Deployment size. The scientific Python stack plus a 70 MB model is ~315 MB. Splitting runtime from training dependencies was needed to fit the 500 MB limit.

Train-serve skew. Any difference between training and inference preprocessing corrupts predictions silently. One saved pipeline removes the possibility.

CHAPTER 6: FUTURE SCOPE

Probability calibration. Fit a calibrator on held-out data and report a reliability diagram and Brier score. The single change that would most improve practical usefulness.

Cost-sensitive thresholds. Replace the F1 criterion with an explicit cost matrix, placing the operating point where expected loss is minimised.

Out-of-time validation. With origination dates, train on an earlier period and test on a later one — the standard approach for credit models, precisely because random splits flatter them.

Better explanations. Permutation importance on held-out data is less biased than the impurity measure used here. Per-prediction attributions (e.g. SHAP) would support the adverse-action explanations regulated lending requires.

Fairness assessment. With the relevant attributes, test for disparate impact across protected groups and mitigate where needed.

The field is moving toward models that are auditable, not merely accurate — regulation increasingly requires that an automated credit decision be explainable to the person it affects.

CHAPTER 7: CONCLUSION

A Random Forest was trained on 150,000 labelled records and evaluated on 30,000 held-out borrowers scored exactly once, reaching ROC-AUC 0.8654 and PR-AUC 0.4014 against a 0.0668 baseline.

The methodological findings matter as much as the numbers:

- Accuracy is actively misleading here — a model that ignores its inputs scores higher than the trained one.
- Class weighting, the conventional remedy, moved the model along the precision-recall curve without lifting it; an explicit threshold selected on validation was used instead.
- Undocumented placeholder codes carried real signal and were handled with missing-value indicators rather than imputed away.
- The residual gap between in-sample and held-out performance was measured and reported, not omitted.

The engineering discipline is what makes the figures trustworthy: preprocessing held inside the saved pipeline so training and inference cannot diverge, the threshold stored with the model so the served cut-off and reported metrics agree, and every documented number generated from the artifact rather than transcribed.

Table 7.1: Technology stack

Layer	Technology
Dataset	Kaggle Give Me Some Credit — 150,000 records
Model	scikit-learn RandomForestClassifier, joblib
Data	pandas, NumPy; matplotlib, seaborn
API	FastAPI, Pydantic, Uvicorn
Frontend	React, TypeScript, Vite, Tailwind CSS
Testing	pytest — 61 tests
Deployment	Vercel (frontend and backend)

The honest conclusion is that this system extracts real, measurable signal from borrower data and is simultaneously nowhere near fit to decide anybody's credit application. Holding both statements at once is the central skill the exercise teaches.

REFERENCES

[1] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] Kaggle and Credit Fusion, “Give Me Some Credit.” Available: https://www.kaggle.com/competitions/GiveMeSomeCredit/data. Accessed: October 24, 2026.

[4] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” PLOS ONE, vol. 10, no. 3, 2015.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[6] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning, 2nd ed. Springer, 2009.

[7] P. Bhalerao, “Loan Default Risk Prediction — source code.” Available: https://github.com/prasadbhalerao1/ML_Project. Accessed: October 24, 2026.

[8] P. Bhalerao, “Loan Delinquency Risk — deployed application.” Available: https://delinquency-risk.vercel.app. Accessed: October 24, 2026.